Teamwork Guide: Git, Files and Rules

How the five of us work together without breaking each other's work (Campus Plus project)

1. The one rule that matters most

A **conflict** happens when two people change the same file at the same time. So we follow one simple rule:

Every file has one owner. Only the owner edits it. If you need a change in someone else's file, ask them (or send a message in the group).

2. Who owns what

Member	Owns (files and features)
Member 1	ER merge, schema runner (run_all.sql), migrations, master data (departments, categories, pickup points), SQL reports, README and docs. Also owns the shared files: server.js, db.js, main.css, package.json
Member 2	Users, login, roles, profile, ratings, in-app notifications
Member 3	Requests, seller items, search and filter, request pages
Member 4	Offers, offer history, offer checks, closing, choosing the winner
Member 5	Transactions, handover, complaints, penalties, admin pages, test checklist, demo prep

3. Folder structure

```
/database
   /schema       01_users.sql  02_catalogue.sql  03_bidding.sql
                 04_deals.sql  05_trust.sql        (one file per module)
   /seed         seed files, one per module
   /migrations   changes made after the ER freeze (numbered files)
   run_all.sql   runs all schema files in order
/backend
   /routes       auth.js  requests.js  offers.js  deals.js
   /controllers  one file per module
   server.js  db.js               (shared: Member 1 only)
/frontend
   /pages        each person's own pages
   /js  /css     one file per module, main.css is shared
/docs            ER diagram, API list, README
```

Put your work only in your own files. Name files after your module so everyone can see who owns what.

4. Database rules

1. **Split the schema.** Each owner writes only their own schema file. Nobody edits another person's SQL file.
2. **Everyone uses a local MySQL.** Do not share one database. Build yours from run_all.sql plus the seed files.
3. **After the ER freeze, no silent changes.** To change a table, tell the group, write it in the Excel "Schema changes" tab, and wait for Member 1 to approve.
4. **Changes go in a new file.** Save it as migrations/002_add_rating_column.sql. Never edit an old file that others have already run.
5. **Use the agreed names.** Tables are plural and lowercase (users, offers). Every table has id as primary key. Foreign keys are named after the table (user_id, request_id).

5. Git rules

- **main always works.** Nobody pushes directly to main.
- **One branch per person per module.** Name it feature/module-name, for example feature/offers.
- **Commit small and often** with a clear message, for example "Add offer validation".
- **Pull every morning** so you start from the latest code.
- **Merge at least every 2 days.** Long-lived branches create big conflicts.
- **Every merge goes through a Pull Request.** One teammate checks it runs, then merges it.
- **Never commit passwords.** Keep secrets in .env (ignored by Git) and share a .env.example with empty values.
- **Installing a package?** Tell the group, because package.json is shared. Pull before and after.

6. Your daily routine

1. Open the project and run `git pull origin main` (get the latest code).
2. Work on your own branch, in your own files.
3. Save often: `git add .` then `git commit -m "short message"`.
4. End of the day: `git push origin your-branch-name`.
5. When a piece works, open a Pull Request and ask a teammate to review it.
6. Update your Status in the Excel tracker.

7. Git commands you need

What you want	Command
Get the project the first time	`git clone <repository-url>`
Make your branch	`git checkout -b feature/offers`
See what changed	`git status`
Save changes	`git add . then git commit -m "message"`
Get latest code	`git pull origin main`
Send your work to GitHub	`git push origin feature/offers`
Switch branch	`git checkout main`

8. If a conflict happens

Do not panic and do not delete other people's work.

1. Git will tell you which file has a conflict. Open it and look for the lines with <<<<<<<, ======= and >>>>>>>.
2. Message the owner of that file and fix it together.
3. Keep the correct lines and delete the marker lines.
4. Run `git add filename` and then `git commit`.
5. Run the app and check that it still works before pushing.

9. Agree before you code

• **API list (Excel tab).** For every route, write the method, what it receives and what it returns. The frontend can use fake data until the real API is ready.

• **Agreed values.** For example, request status = open, closed, cancelled. Payment status = pending, paid, failed. Everyone uses exactly these words.

• **Excel tracker.** Update your task Status every day (Not started, In progress, In review, Done). This is how we measure progress.

10. Docker (optional, MySQL only)

One member will try Docker for the MySQL database in Phase 0, for 1-2 hours. We use it only if it works on everyone's laptop. If not, we drop it and use normal MySQL. The schema and seed files must work with both. We do not put the Node app in Docker.

11. Do and Don't

DO	DON'T
Edit only your own files Pull every morning Commit small and often Use Pull Requests Tell the group before changing a table Update the Excel daily Ask when stuck	Push directly to main Edit someone else's file without asking Change a table silently Commit passwords or node_modules Keep a branch for a week without merging Work for days without telling anyone

12. Phase 0 checklist (Days 1-3)

Done	Task	Owner
[ ]	Meeting decisions written down and shared	Member 1
[ ]	Own part of the ER draft sent to Member 1	Everyone
[ ]	ER merged, reviewed and frozen	Member 1 + all
[ ]	GitHub repository, folders, README created. main branch protected	Member 1
[ ]	Excel tracker and API list created	Member 5
[ ]	Node, MySQL, Git and Postman installed. A "hello" API runs locally	Everyone
[ ]	Everyone has cloned the repo and made their own branch	Everyone
[ ]	Docker MySQL trial (1-2 hours). Adopt only if it works for everyone, otherwise drop it	Member who suggested it

Checkpoint 0 is passed when: the ER is frozen, the schema scripts run without errors, and everyone can run the project on their own laptop.